

Lab 2: POSIX Threads (pthreads) in C - Detailed Study Guide

This comprehensively expanded guide explains how to write shared-memory parallel programs in C using the POSIX Threads (**pthread**s) library. It includes inline examples, concept explanations, and solved mock exam questions to help you prepare.

1. Core Concepts & Thread Creation

What is a Thread?

A thread is the smallest sequence of programmed instructions that can be managed independently by a scheduler. Unlike separate processes (which have isolated memory spaces), threads within the *same process* **share the same address space, global variables, and heap memory**.

How to Create Threads (**pthread_create**)

The foundational function for starting a new thread is **pthread_create**. Here is its signature:

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                  void *(*start_routine) (void *), void *arg);
```

- **thread**: Pointer to a thread identifier (an integer ID).
- **attr**: Attributes for the thread (e.g., stack size, scheduling). Use **NULL** for defaults.
- **start_routine**: A pointer to the C function the thread will execute. MUST take **void*** and return **void***.
- **arg**: The argument passed to the function. If you need multiple arguments, pack them in a **struct**.

How to Wait for Threads (**pthread_join**)

When the main thread finishes its setup, it shouldn't just exit. If the main thread dies, it kills all child threads. You must wait for them using:

```
int pthread_join(pthread_t thread, void **retval);
```

💡 Example: Basic Thread Usage

```
#include <stdio.h>
#include <pthread.h>

// The function the thread runs
void* say_hello(void* arg) {
```

```

    int id = *((int*)arg);
    printf("Hello from thread %d!\n", id);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    int id1 = 1, id2 = 2; // Pass local copies of data, NOT loop counters directly

    pthread_create(&t1, NULL, say_hello, &id1);
    pthread_create(&t2, NULL, say_hello, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}

```

Compilation: `gcc program.c -o program -pthread`

2. Shared Data, Race Conditions, and Synchronization

Since threads share the same global/heap variables, what happens when multiple threads try to modify the same variable at exactly the same time?

The Race Condition

A race condition occurs when two threads read and write to shared data concurrently, and the final state depends on the non-deterministic scheduling of threads.

Example of failure (`counter++`):

1. Thread A reads `counter` (value: 5)
2. Thread B reads `counter` (value: 5)
3. Thread A increments and writes back `counter = 6`
4. Thread B increments and writes back `counter = 6` (*Expected 7, got 6!*)

Primitive 1: Mutexes (Mutual Exclusion)

A mutex acts as a lock. A thread must "lock" the mutex before touching shared data. If another thread currently holds the lock, the new thread **blocks/sleeps** until the lock is released.

```

#include <pthread.h>
int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* increment(void* arg) {
    for(int i=0; i<10000; i++) {
        pthread_mutex_lock(&lock);    // ENTRY SECTION
        counter++;                    // CRITICAL SECTION
        pthread_mutex_unlock(&lock);  // EXIT SECTION
    }
}

```

```

    }
    return NULL;
}

```

Primitive 2: Semaphores (<semaphore.h>)

While a mutex allows exactly *one* thread into the critical section, a Counting Semaphore allows up to N threads. It's built on a counter: N slots available. `sem_wait()` decreases the counter. `sem_post()` increases it. If it hits 0, `sem_wait()` blocks the thread.

When to use? When you have identical resources (e.g., 5 database connections, 3 print server slots).

```

#include <semaphore.h>
sem_t slots;

// Inside main: Initialize to 3 available slots
sem_init(&slots, 0, 3);

// Inside thread:
sem_wait(&slots);
// ... Use one of the 3 slots ...
sem_post(&slots);

```

Primitive 3: Atomic Operations (<stdatomic.h>)

If you only need to update a simple variable (like a counter or boolean flag), mutexes are too slow. Atomic operations rely on CPU-level hardware instructions to increment a value indivisibly. It never sleeps/blocks.

```

#include <stdatomic.h>
atomic_int counter = 0;

void* increment(void* arg) {
    atomic_fetch_add(&counter, 1); // Indivisible, hardware-level safe increment!
    return NULL;
}

```

3. Advanced Thread Patterns: Deadlocks and Livelocks

- **Deadlock:** Thread A holding Mutex 1 waits forever for Mutex 2. Meanwhile, Thread B holding Mutex 2 waits forever for Mutex 1. Everything freezes.
 - *Solution:* Always lock multiple mutexes in the exact same strict order across all threads.
 - **Livelock:** Threads try to avoid deadlock by acquiring Mutex 1, failing to acquire Mutex 2, releasing Mutex 1, sleeping, and retrying. They loop forever yielding to each other but making 0 progress.
-

4. Exam Cheat Sheet: Adapting Thread Code

When the question asks for a specific behavior, here is exactly what you change in your standard pthread skeleton:

If the question says...	You should use/change...	Code Snippet
"Threads are updating a single shared counter"	Use an <code>atomic</code> variable or <code>pthread_mutex_t</code> .	<code>atomic_fetch_add(&counter, 1);</code>
"Threads must not exceed N simultaneous connections (e.g. print slots)"	Use a Counting Semaphore (<code>sem_t</code>) initialized to \$N\$.	In main: <code>sem_init(&sem, 0, N);</code> In thread: <code>sem_wait(&sem); ... sem_post(&sem);</code>
"Each thread needs a unique ID (0, 1, 2...)"	Pass by value casting the integer, NOT by reference.	<code>pthread_create(&t, NULL, func, (void*)(long)i);</code>
"Threads must wait for each other at a certain checkpoint"	Depending on the context, use a POSIX Barrier (if available) or wait logic using condition variables.	<code>pthread_barrier_wait(&barrier);</code>
"Return a calculated value from the thread"	Dynamically allocate the result and <code>return</code> it. Catch it with <code>pthread_join</code> .	Thread: <code>int* res = malloc(sizeof(int)); *res = 42; return res;</code> Main: <code>void* ptr; pthread_join(t, &ptr);</code>

5. Multi-Layer Exam Preparation: Solved Mock Questions

Below are high-probability exam questions and step-by-step solutions demonstrating how to apply these concepts.

Question A: The Buggy Loop Variable (Concept: Memory Addresses)

Problem Statement: A student wrote the following code to spawn 4 threads. Each thread should print its ID (0, 1, 2, 3). However, the output is non-deterministic and often prints "Thread 3, Thread 3, Thread 3, Thread 4". Explain why and fix it.

```
int num_threads = 4;
pthread_t threads[num_threads];
for(int i = 0; i < num_threads; i++) {
    pthread_create(&threads[i], NULL, print_id, &i);
}
```

Step-by-step Solution:

- 1. Identify the bug:** We are passing the memory address of the loop variable `&i` to all threads. Because `pthread_create` is asynchronous, the `for` loop continues executing and modifies `i` before the child threads wake up and read it.
- 2. Fix Method 1 (Malloc):** Allocate unique heap memory for each thread's argument.

```

for(int i = 0; i < num_threads; i++) {
    int* id_ptr = malloc(sizeof(int));
    *id_ptr = i;
    pthread_create(&threads[i], NULL, print_id, id_ptr);
}
// Inside thread: int id = *(int*)arg; free(arg);

```

3. **Fix Method 2 (Pass by Value via Cast):** Cast the integer value directly into the `void*` pointer struct.

```

for(long i = 0; i < num_threads; i++) { // Using long because pointers are
64-bit
    pthread_create(&threads[i], NULL, print_id, (void*)i);
}
// Inside thread: long id = (long)arg;

```

Question B: The Concurrent Bank Accounts (Concept: Double Locks/Deadlocks)

Problem Statement: Design a concurrent function `transfer(int* from_acc, int* to_acc, int amount, pthread_mutex_t* lock_from, pthread_mutex_t* lock_to)`. It must safely deduct `amount` from `from_acc` and add it to `to_acc`. How do you protect this from race conditions, and how do you ensure it does not deadlock if another thread tries a reverse transfer simultaneously?

Step-by-step Solution:

1. **Race Condition Prevention:** Both accounts must be locked before performing the read/write operations.
2. **Deadlock Prevention:** If Thread 1 calls `transfer(A, B)` and Thread 2 calls `transfer(B, A)`, Thread 1 locks A, Thread 2 locks B. They both freeze waiting for the other.
3. **Impose an Ordering:** We must always lock the mutexes in the same order, regardless of which account is "from" and which is "to". The easiest way in C is comparing the memory addresses of the locks.

```

void transfer(int* from, int* to, int amount, pthread_mutex_t* mtx_from,
pthread_mutex_t* mtx_to) {
    // 1. Determine strict lock order based on memory address!
    pthread_mutex_t* first_lock = (mtx_from < mtx_to) ? mtx_from : mtx_to;
    pthread_mutex_t* second_lock = (mtx_from < mtx_to) ? mtx_to : mtx_from;

    // 2. Lock in order
    pthread_mutex_lock(first_lock);
    pthread_mutex_lock(second_lock);

    // 3. Perform atomic-like operation
    *from -= amount;
    *to += amount;

    // 4. Unlock in reverse order
    pthread_mutex_unlock(second_lock);
}

```

```
    pthread_mutex_unlock(first_lock);
}
```

Question C: The Producer-Consumer Histogram (Concept: Chunking & Mutex Scope)

Problem Statement: You have a massive array of 100,000 random numbers (0-9). Use 4 threads to count the frequency of each number. Output the final histogram array `int freq[10]`.

Step-by-step Solution:

1. **Bad Design (Global Mutex):** Wrapping a global mutex around `freq[number]++` inside the innermost loop is atrocious for performance—it will run slower than a sequential program due to constant lock contention.
2. **Good Design (Thread-Local Reduction):** Each thread should aggregate a private, local histogram array. Only when a thread finishes processing its entire chunk, it locks a global mutex to merge its private array into the global `freq` array.

```
#include <stdio.h>
#include <pthread.h>

int numbers[100000];
int global_freq[10] = {0};
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;

typedef struct {
    int start_index;
    int end_index;
} Chunk;

void* build_histogram(void* arg) {
    Chunk* chunk = (Chunk*)arg;
    int local_freq[10] = {0}; // Thread-local, no locks needed!

    // 1. Process block entirely locally
    for(int i = chunk->start_index; i < chunk->end_index; i++) {
        local_freq[numbers[i]]++;
    }

    // 2. Merge back to global with ONE lock at the end
    pthread_mutex_lock(&mtx);
    for(int i = 0; i < 10; i++) {
        global_freq[i] += local_freq[i];
    }
    pthread_mutex_unlock(&mtx);

    return NULL;
}
```

Exam Tip: Mentioning "Chunking" and "Thread-Local Aggregation" in written answers frequently earns top marks in concurrent programming exams!